

UBWC ENC/DEC 软件使用说明

整理时间：2026-05-17。说明软件首次配置、后续帧配置、中断处理、时钟域约束和 DEC OTF timing 参数填写方式。

基础约定	时钟域	Bank SRAM
配置频率	ENC 使用流程	DEC 使用流程
DEC OTF timing	状态与中断	回归基线

1. 基础约定

APB 寄存器为 32 bit 宽，地址按 4 byte 对齐。所有 64 bit base address 都按低 32 bit、高 32 bit 的顺序写入。

格式	编码	tile_w	tile_h	说明
RGBA8888	0	16	4	单平面 RGBA, 4 byte/pixel
RGBA1010102	1	16	4	单平面 RGBA, 4 byte/pixel
YUV420_8 / NV12	2	32	8	Y + UV 双平面
YUV420_10 / P010	3	32	4	Y + UV 双平面, 16 bit component 存储

连续帧模式下，如果格式、分辨率、tile layout 和 OTF timing 不变，软件通常只需要每帧更新 UBWC buffer base address。

2. 时钟域约束

当前 wrapper 级回归使用 OTF 320 MHz、core/VIVO 200 MHz、AXI 500 MHz。APB 只负责寄存器访问，视频数据通路不依赖 APB 时钟。

时钟域	频率	周期	ENC 端口	DEC 端口	主要逻辑
APB clock	100 MHz	10.000 ns	PCLK	PCLK	APB 寄存器访问
AXI/control clock	500 MHz	2.000 ns	i_axi_clk	i_axi_clk	AXI 读写、APB 同步后的控制、状态统计
core/VIVO clock	200 MHz	5.000 ns	i_vivo_clk	i_vivo_clk	VIVO encode/decode core
OTF clock	320 MHz	3.125 ns	i_otf_clk	i_otf_clk	OTF 输入或输出接口

AXI/control、core/VIVO 和 OTF 之间必须通过异步 FIFO 或明确 CDC 逻辑隔离。不要把多 bit payload/counter 直接打两拍跨时钟域。

3. Bank0/Bank1 SRAM 容量要求

ENC 和 DEC wrapper 都把 bank0/bank1 SRAM 留在模块外部实现。两个 bank 的接口都是 128 bit 数据宽度，即每个地址 word 为 16 byte。bank0 和 bank1 是同规格的工作 buffer，软件不需要配置它们的大小，但集成顶层时需要按下面容量提供 SRAM。

模块	bank	当前 RTL 参数/端口	按当前默认参数需要的 RAM	说明
ENC	bank0	COM_BUF_AW=12 , COM_BUF_DW=128	4096 x 128 bit = 64 KiB = 512 Kbit	与 bank1 对称，是 line-to-tile 的 ping-pong 工作区，不固定对应 Y/UV。
ENC	bank1	COM_BUF_AW=12 , COM_BUF_DW=128	4096 x 128 bit = 64 KiB = 512 Kbit	与 bank0 对称。当前默认 ENC 两个 bank 合计 128 KiB。
DEC	bank0	COM_BUF_AW=12 , COM_BUF_DW=128	4096 x 128 bit = 64 KiB =	与 bank1 对称，是 tile-to-OTF 的 ping-pong 工作

模块	bank	当前 RTL 参数/端口	按当前默认参数需要的 RAM	说明
			512 Kbit	区，不固定对应 Y/UV。
DEC	bank1	COM_BUF_AW=12 , COM_BUF_DW=128	4096 x 128 bit = 64 KiB = 512 Kbit	与 bank0 对称。当前默认 DEC 两个 bank 合计 128 KiB。

当前默认 `COM_BUF_AW=12` 覆盖 4096 宽 RGBA 与 YUV420 场景。4096 宽 YUV420 使用两个物理 bank 组成一个逻辑工作组：bank0 保存 Y0..7 与 UV0..3，bank1 保存 Y8..15 与 UV4..7。因此每个 bank 固定 4096 x 128 bit 即可。这里的 SRAM 是 wrapper 外部的临时工作 RAM，不是压缩后 UBWC frame buffer。

3.1 ENC bank 容量计算

ENC 的 bank0/bank1 用在 `ubwc_enc_line_to_tile`，作用是把 OTF 输入行数据重排成 tile 输出。两个 bank 轮流写入/读出，所以每个 bank 都要能容纳一个完整工作组。

```
// 128 bit word, 1 word = 16 byte
y_tile_cols = ceil_div(aligned_width_px, tile_w)
uv_tile_cols = 0 for RGBA; otherwise y_tile_cols

// 每个 ENC bank 的最小 word 数
RGBA8888 / RGBA10:
    enc_bank_words = y_tile_cols * 16

YUV420_8 / NV12:
YUV420_10 / P010:
    enc_bank_words = (y_tile_cols + uv_tile_cols) * 16
    // 偶数 Y tile-row 和对应 UV tile-row 放在同一个 bank
    // 通常 uv_tile_cols == y_tile_cols, 所以等价于 y_tile_cols * 32

// 参数化实现时
enc_bank_addr_width = ceil_log2(enc_bank_words)
enc_bank_bytes      = 2^enc_bank_addr_width * 16
```

最大宽度 场景	格式	y_tile_cols	最小有效 word 数	建议地址 宽度	每个 bank 容量
4096 px	RGBA8888/RGBA10	256	4096	12 bit	64 KiB
4096 px	YUV420_8/NV12	128	4096	12 bit	64 KiB
4096 px	YUV420_10/P010	128	4096	12 bit	64 KiB

因此当前 ENC 最大 4096 宽配置统一使用 `COM_BUF_AW=12`，每个 bank 64 KiB，不再为 YUV420 增加到 13 bit。

3.2 DEC bank 容量计算

DEC 的 bank0/bank1 用在 `ubwc_dec_tile_to_otf`，作用是把 VIVO 解出的 tile 数据重组成 OTF 输出行。外部 bank 容量由顶层参数 `COM_BUF_AW` 决定。

```
// DEC 地址空间, 128 bit word
dec_bank_words = 2 ^ COM_BUF_AW
dec_bank_bytes = dec_bank_words * 16

// COM_BUF_AW=12 时, 4096 像素宽度下的有效地址使用范围
RGBA8888 / RGBA10:
    one bank logical group:
        word 0 .. 4095

YUV420_8 / NV12:
YUV420_10 / P010:
    two-bank logical group:
        bank0 word 0      .. 2047 : Y   row 0 .. 7
        bank0 word 2048 .. 3071 : UV  row 0 .. 3
        bank0 word 3072 .. 4095 : unused/reserved
        bank1 word 0      .. 2047 : Y   row 8 .. 15
        bank1 word 2048 .. 3071 : UV  row 4 .. 7
        bank1 word 3072 .. 4095 : unused/reserved
```

4096 宽 YUV420 会同时占用两个 bank，两个 bank 组成一个逻辑工作组，当前工作组读完后一起释放。软件不需要配置 bank 选择，只需要按每帧写入 UBWC buffer 地址。

4. 首次配置和后续配置

场景	软件动作
首次配置	读取 VERSION/DATE；使能中断；配置格式、尺寸、tile layout、CI/VIVO、metadata geometry、DEC OTF timing；写入第一帧 ENC/DEC 地址。
后续帧配置	如果格式、尺寸、layout、timing 不变，只写新一帧的 base address；ENC/DEC 都需要在本帧地址组写完整后写 <code>IRQ_CTRL[5]=1</code> 作为 START token。
重新配置静态项	只有格式、分辨率、layout 或 DEC OTF timing 变化时才重写静态项；应先确认当前帧处理完成，再切换静态配置。

4.1 ENC 配置清单

寄存器地址	寄存器功能	配置模式	默认配置/说明
0x000	VERSION	上电后读取一次	只读
0x004	DATE	上电后读取一次	只读
0x060[0]	IRQ enable	上电后或中断策略变化时配置	1
0x00C	REG_TILE_CFG1: 4-line/lossy/tile_pitch	更改图像格式时配置	按格式计算
0x008	REG_TILE_CFG0: ubwc/bank/swizzle	更改图像格式	按系统策略

寄存器地址	寄存器功能	配置模式	默认配置/说明
		时配置	
0x014	REG_ENC_CI_CFG1: lossy	更改图像格式 时配置	0 或按格式
0x018	REG_ENC_CI_CFG2: reserved cfg	更改图像格式 时配置	0
0x01C	REG_ENC_CI_CFG3: reserved cfg	更改图像格式 时配置	0
0x010	REG_ENC_CI_CFG0: input_type/alen	更改图像格式 时配置	input_type=1, alen=7
0x024	REG_OTF_CFG1: width/height	更改图像格式 时配置	按图像尺寸
0x028	REG_OTF_CFG2: tile_w/tile_h	更改图像格式 时配置	RGBA=16x4; YUV420_8=32x8; YUV420_10=32x4
0x02C	REG_OTF_CFG3: tile columns	更改图像格式 时配置	[15:0] y_tile_cols; [31:16] uv_tile_cols, RGBA 的 uv 写 0
0x050	REG_META_ACTIVE_SIZE	更改图像格式 时配置	按图像尺寸
0x054	REG_META_PITCH	更改图像格式 时配置	按格式/宽度计算

寄存器地址	寄存器功能	配置模式	默认配置/说明
0x020	REG_OTF_CFG0: format	更改图像格式时配置	0/1/2/3
0x030	REG_META_BASE_Y_LO	每次配置	压缩后 Y metadata 存储基地址低 32 bit; RGBA 使用该槽位
0x034	REG_META_BASE_Y_HI	每次配置	压缩后 Y metadata 存储基地址高 32 bit; RGBA 使用该槽位
0x038	REG_TILE_BASE_Y_LO	每次配置	压缩后 Y 压缩数据存储基地址低 32 bit; RGBA 使用该槽位
0x03C	REG_TILE_BASE_Y_HI	每次配置	压缩后 Y 压缩数据存储基地址高 32 bit; RGBA 使用该槽位
0x040	REG_META_BASE_UV_LO	每次配置	压缩后 UV metadata 存储基地址低 32 bit; RGBA 写 0
0x044	REG_META_BASE_UV_HI	每次配置	压缩后 UV metadata 存储基地址高 32 bit; RGBA 写 0
0x048	REG_TILE_BASE_UV_LO	每次配置	压缩后 UV 压缩数据存储基地址低 32 bit; RGBA 写 0
0x04C	REG_TILE_BASE_UV_HI	每次配置	压缩后 UV 压缩数据存储基地址高 32 bit; 写完四个地址后写 IRQ_CTRL[5] 启动本帧

4.2 DEC 配置清单

寄存器地址	寄存器功能	配置模式	默认配置/说明
0x000	VERSION	上电后读取一次	只读

寄存器地址	寄存器功能	配置模式	默认配置/说明
0x004	DATE	上电后读取一次	只读
0x060[0]	IRQ enable	上电后或中断策略变化时配置	1
0x008	APB_ADDR_TILE_CFG0: swizzle/layout	更改图像格式时配置	按系统策略
0x00C	APB_ADDR_TILE_CFG1: tile pitch	更改图像格式时配置	按格式/宽度计算
0x010	APB_ADDR_TILE_CFG2: CI cfg	更改图像格式时配置	input_type=1, 其余按格式
0x014	APB_ADDR_VIVO_CFG	更改图像格式时配置	vivo_ubwc_en=1
0x018	APB_ADDR_OTF_CFG0: img_width/format	更改图像格式时配置	按输出图像配置
0x01C	APB_ADDR_OTF_CFG1: h_total/h_sync	更改图像格式时配置	按 OTF timing 配置
0x020	APB_ADDR_OTF_CFG2: h_bp/h_act	更改图像格式时配置	按 OTF timing 配置
0x024	APB_ADDR_OTF_CFG3: v_total/v_sync	更改图像格式时配	按 OTF timing 配置

寄存器地址	寄存器功能	配置模式	默认配置/说明
		置	
0x028	APB_ADDR_OTF_CFG4: v_bp/v_act	更改图像 格式时配 置	按 OTF timing 配置
0x02C	APB_ADDR_META_CFG0: tile_x/y	更改图像 格式时配 置	按图像尺寸计算；YUV420 的 UV tile 数内部推导
0x030	REG_META_BASE_Y_LO	每次配置	RGBA/Y PLANE metadata 读 取基地址低 32 bit
0x034	REG_META_BASE_Y_HI	每次配置	RGBA/Y PLANE metadata 读 取基地址高 32 bit
0x038	REG_TILE_BASE_Y_LO	每次配置	RGBA/Y PLANE 压缩数据读取 基地址低 32 bit
0x03C	REG_TILE_BASE_Y_HI	每次配置	RGBA/Y PLANE 压缩数据读取 基地址高 32 bit
0x040	REG_META_BASE_UV_LO	每次配置	UV PLANE metadata 读取基 地址低 32 bit
0x044	REG_META_BASE_UV_HI	每次配置	UV PLANE metadata 读取基 地址高 32 bit
0x048	REG_TILE_BASE_UV_LO	每次配置	UV PLANE 压缩数据读取基址 地址低 32 bit；RGBA 图像不关 心
0x04C	REG_TILE_BASE_UV_HI	每次配置	UV PLANE 压缩数据读取基址 地址高 32 bit；RGBA 图像不关 心
0x060[5]	START	每次配置	地址组写完整后写 1；地址写 入只填充队列，不单独启动帧

4.3 ENC 分辨率参考配置

下面只列 ENC 静态几何相关寄存器，覆盖 RGBA8888、YUV420_8/NV12、YUV420_10/P010。每帧 base address 仍按实际输出 buffer 地址单独写入。

格式	分辨率	关键计算	ENC 写值参考
RGBA8888	128x128	tile=16x4, bpp=4 Y cols/rows=8/32 UV cols/rows=0/0 stored Y/UV=128/0 tile_pitch=32, meta_pitch=64 size Y(meta/tile)=0x1000/0x10000 size UV(meta/tile)=0x0/0x0	0x00C REG_TILE_CFG1 = 0x00200001 0x024 REG_OTF_CFG1 = 0x00800080 0x028 REG_OTF_CFG2 = 0x00040010 0x02C REG_OTF_CFG3 = 0x00000008 0x050 REG_META_ACTIVE_SIZE= 0x00800080 0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000000
RGBA8888	544x1200	tile=16x4, bpp=4 Y cols/rows=36/300 UV cols/rows=0/0 stored Y/UV=1200/0 tile_pitch=144, meta_pitch=64 size Y(meta/tile)=0x5000/0x2A3000 size UV(meta/tile)=0x0/0x0	0x00C REG_TILE_CFG1 = 0x00900001 0x024 REG_OTF_CFG1 = 0x04B00220 0x028 REG_OTF_CFG2 = 0x00040010 0x02C REG_OTF_CFG3 = 0x00000024 0x050 REG_META_ACTIVE_SIZE= 0x04B00220 0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000000

格式	分辨率	关键计算	ENC 写值参考
RGBA8888	1920x1080	tile=16x4, bpp=4 Y cols/rows=120/272 UV cols/rows=0/0 stored Y/UV=1088/0 tile_pitch=480, meta_pitch=128 size Y(meta/tile)=0x9000/0x7F8000 size UV(meta/tile)=0x0/0x0	0x00C REG_TILE_CFG1 = 0x01E00001 0x024 REG_OTF_CFG1 = 0x04380780 0x028 REG_OTF_CFG2 = 0x00040010 0x02C REG_OTF_CFG3 = 0x00000078 0x050 REG_META_ACTIVE_SIZE= 0x04380780 0x054 REG_META_PITCH = 0x00000080 0x020 REG_OTF_CFG0 = 0x00000000
RGBA8888	4096x600	tile=16x4, bpp=4 Y cols/rows=256/152 UV cols/rows=0/0 stored Y/UV=608/0 tile_pitch=1024, meta_pitch=256 size Y(meta/tile)=0xA000/0x980000 size UV(meta/tile)=0x0/0x0	0x00C REG_TILE_CFG1 = 0x04000001 0x024 REG_OTF_CFG1 = 0x02581000 0x028 REG_OTF_CFG2 = 0x00040010 0x02C REG_OTF_CFG3 = 0x00000100 0x050 REG_META_ACTIVE_SIZE= 0x02581000 0x054 REG_META_PITCH = 0x00000100 0x020 REG_OTF_CFG0 = 0x00000000
YUV420_8 / NV12	128x128	tile=32x8, bpp=1 Y cols/rows=4/16 UV cols/rows=4/8 stored Y/UV=128/64 tile_pitch=8, meta_pitch=64	0x00C REG_TILE_CFG1 = 0x00080000 0x024 REG_OTF_CFG1 = 0x00800080 0x028 REG_OTF_CFG2 =

格式	分辨率	关键计算	ENC 写值参考
		size Y(meta/tile)=0x1000/0x4000 size UV(meta/tile)=0x1000/0x2000	0x00080020 0x02C REG_OTF_CFG3 = 0x00040004 0x050 REG_META_ACTIVE_SIZE= 0x00800080 0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000002
YUV420_8 / NV12	544x1200	tile=32x8, bpp=1 Y cols/rows=20/152 UV cols/rows=20/76 stored Y/UV=1216/608 tile_pitch=40, meta_pitch=64 size Y(meta/tile)=0x3000/0xBE000 size UV(meta/tile)=0x2000/0x5F000	0x00C REG_TILE_CFG1 = 0x00280000 0x024 REG_OTF_CFG1 = 0x04B00220 0x028 REG_OTF_CFG2 = 0x00080020 0x02C REG_OTF_CFG3 = 0x00140014 0x050 REG_META_ACTIVE_SIZE= 0x04B00220 0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000002
YUV420_8 / NV12	1920x1080	tile=32x8, bpp=1 Y cols/rows=60/136 UV cols/rows=60/68 stored Y/UV=1088/544 tile_pitch=120, meta_pitch=64 size Y(meta/tile)=0x3000/0x1FE000 size UV(meta/tile)=0x2000/0xFF000	0x00C REG_TILE_CFG1 = 0x00780000 0x024 REG_OTF_CFG1 = 0x04380780 0x028 REG_OTF_CFG2 = 0x00080020 0x02C REG_OTF_CFG3 = 0x003C003C 0x050 REG_META_ACTIVE_SIZE= 0x04380780

格式	分辨率	关键计算	ENC 写值参考
			0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000002
YUV420_8 / NV12	4096x600	tile=32x8, bpp=1 Y cols/rows=128/76 UV cols/rows=128/40 stored Y/UV=608/320 tile_pitch=256, meta_pitch=128 size Y(meta/tile)=0x3000/0x260000 size UV(meta/tile)=0x2000/0x140000	0x00C REG_TILE_CFG1 = 0x01000000 0x024 REG_OTF_CFG1 = 0x02581000 0x028 REG_OTF_CFG2 = 0x00080020 0x02C REG_OTF_CFG3 = 0x00800080 0x050 REG_META_ACTIVE_SIZE= 0x02581000 0x054 REG_META_PITCH = 0x00000080 0x020 REG_OTF_CFG0 = 0x00000002
YUV420_10 / P010	128x128	tile=32x4, bpp=2 Y cols/rows=4/32 UV cols/rows=4/16 stored Y/UV=128/64 tile_pitch=16, meta_pitch=64 size Y(meta/tile)=0x1000/0x8000 size UV(meta/tile)=0x1000/0x4000	0x00C REG_TILE_CFG1 = 0x00100000 0x024 REG_OTF_CFG1 = 0x00800080 0x028 REG_OTF_CFG2 = 0x00040020 0x02C REG_OTF_CFG3 = 0x00040004 0x050 REG_META_ACTIVE_SIZE= 0x00800080 0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000003

格式	分辨率	关键计算	ENC 写值参考
YUV420_10 / P010	544x1200	tile=32x4, bpp=2 Y cols/rows=20/300 UV cols/rows=20/152 stored Y/UV=1200/608 tile_pitch=80, meta_pitch=64 size Y(meta/tile)=0x5000/0x177000 size UV(meta/tile)=0x3000/0xBE000	0x00C REG_TILE_CFG1 = 0x00500000 0x024 REG_OTF_CFG1 = 0x04B00220 0x028 REG_OTF_CFG2 = 0x00040020 0x02C REG_OTF_CFG3 = 0x00140014 0x050 REG_META_ACTIVE_SIZE= 0x04B00220 0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000003
YUV420_10 / P010	1920x1080	tile=32x4, bpp=2 Y cols/rows=60/272 UV cols/rows=60/136 stored Y/UV=1088/544 tile_pitch=240, meta_pitch=64 size Y(meta/tile)=0x5000/0x3FC000 size UV(meta/tile)=0x3000/0x1FE000	0x00C REG_TILE_CFG1 = 0x00F00000 0x024 REG_OTF_CFG1 = 0x04380780 0x028 REG_OTF_CFG2 = 0x00040020 0x02C REG_OTF_CFG3 = 0x003C003C 0x050 REG_META_ACTIVE_SIZE= 0x04380780 0x054 REG_META_PITCH = 0x00000040 0x020 REG_OTF_CFG0 = 0x00000003
YUV420_10 / P010	4096x600	tile=32x4, bpp=2 Y cols/rows=128/152 UV cols/rows=128/76 stored Y/UV=608/304 tile_pitch=512, meta_pitch=128	0x00C REG_TILE_CFG1 = 0x02000000 0x024 REG_OTF_CFG1 = 0x02581000 0x028 REG_OTF_CFG2 =

格式	分辨率	关键计算	ENC 写值参考
		size Y(meta/tile)=0x5000/0x4C0000 size UV(meta/tile)=0x3000/0x260000	0x00040020 0x02C REG_OTF_CFG3 = 0x00800080 0x050 REG_META_ACTIVE_SIZE= 0x02581000 0x054 REG_META_PITCH = 0x00000080 0x020 REG_OTF_CFG0 = 0x00000003

4.3.1 ENC 地址和 pitch 计算方法

```
// 基础函数
ceil_div(a, b) = (a + b - 1) / b
align_up(a, n) = ceil_div(a, n) * n

// 格式参数
RGBA8888:          format=0, tile_w=16, tile_h=4, bpp=4
YUV420_8 / NV12:    format=2, tile_w=32, tile_h=8, bpp=1
YUV420_10 / P010:   format=3, tile_w=32, tile_h=4, bpp=2

// ENC pitch / tile 数
aligned_width_px    = align_up(width_px, tile_w * 4)
y_tile_cols         = ceil_div(aligned_width_px, tile_w)
uv_tile_cols        = 0 for RGBA; otherwise y_tile_cols
stored_y_height      = align_up(height_px, tile_h * 4)
stored_uv_height     = align_up(ceil_div(height_px, 2), tile_h * 4)
surface_pitch_bytes  = align_up(width_px * bpp, tile_w * 4 * bpp)
tile_pitch           = surface_pitch_bytes / 16
meta_pitch           = align_up(y_tile_cols, 64)

// ENC metadata / tile size
y_tile_rows          = ceil_div(stored_y_height, tile_h)
uv_tile_rows          = ceil_div(stored_uv_height, tile_h)
meta_y_size           = align_up(meta_pitch * align_up(y_tile_rows, 16), 4096)
tile_y_size           = align_up(surface_pitch_bytes * stored_y_height, 4096)
```

```
meta_uv_size      = 0 for RGBA; otherwise align_up(meta_pitch * align_
tile_uv_size      = 0 for RGBA; otherwise align_up(surface_pitch_bytes

// 连续输出 buffer 地址, base 是 Y/RGBA metadata 起始地址
REG_META_BASE_Y   = base
REG_TILE_BASE_Y   = REG_META_BASE_Y + meta_y_size
REG_META_BASE_UV  = 0 for RGBA; otherwise REG_TILE_BASE_Y + tile_y_siz
REG_TILE_BASE_UV  = 0 for RGBA; otherwise REG_META_BASE_UV + meta_uv_s
total_size        = meta_y_size + tile_y_size + meta_uv_size + tile_uv
```

4.4 DEC 分辨率参考配置

下面只列 DEC 静态几何和 OTF timing 相关寄存器，覆盖 RGBA8888、YUV420_8/NV12、YUV420_10/P010。OTF 示例默认 HSA=4、HBP=8、HFP=20、VSA=2、VBP=4、VFP=6；如果系统 timing 不同，只替换 OTF_CFG1..4。

格式	分辨率	关键计算	DEC 写值参考
RGBA8888	128x128	tile=16x4, bpp=4 Y meta cols/rows=8/32 UV meta cols/rows=0/0 stored Y/UV=128/0 tile_pitch=32, meta_pitch=64 size Y(meta/tile)=0x1000/0x10000 size UV(meta/tile)=0x0/0x0	0x008 TILE_CFG0 = 0x00000706 0x00C TILE_CFG1 = 0x00000020 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00000080 0x01C OTF_CFG1 = 0x000400A0 0x020 OTF_CFG2 = 0x00800008 0x024 OTF_CFG3 = 0x0002008C 0x028 OTF_CFG4 = 0x00800004
			0x02C META_CFG0 = 0x00200008
RGBA8888	544x1200	tile=16x4, bpp=4 Y meta cols/rows=36/300	0x008 TILE_CFG0 = 0x00000706

格式	分辨率	关键计算	DEC 写值参考
		UV meta cols/rows=0/0 stored Y/UV=1200/0 tile_pitch=144, meta_pitch=64 size Y(meta/tile)=0x5000/0x2A3000 size UV(meta/tile)=0x0/0x0	0x00C TILE_CFG1 = 0x00000090 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00000220 0x01C OTF_CFG1 = 0x00040240 0x020 OTF_CFG2 = 0x02200008 0x024 OTF_CFG3 = 0x000204BC 0x028 OTF_CFG4 = 0x04B00004 0x02C META_CFG0 = 0x012C0024
RGBA8888	1920x1080	tile=16x4, bpp=4 Y meta cols/rows=120/272 UV meta cols/rows=0/0 stored Y/UV=1088/0 tile_pitch=480, meta_pitch=128 size Y(meta/tile)=0x9000/0x7F8000 size UV(meta/tile)=0x0/0x0	0x008 TILE_CFG0 = 0x00000706 0x00C TILE_CFG1 = 0x000001E0 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00000780 0x01C OTF_CFG1 = 0x000407A0 0x020 OTF_CFG2 = 0x07800008 0x024 OTF_CFG3 = 0x00020444 0x028 OTF_CFG4 = 0x04380004 0x02C META_CFG0 = 0x01100078
RGBA8888	4096x600	tile=16x4, bpp=4 Y meta cols/rows=256/152	0x008 TILE_CFG0 = 0x00000706

格式	分辨率	关键计算	DEC 写值参考
		UV meta cols/rows=0/0 stored Y/UV=608/0 tile_pitch=1024, meta_pitch=256 size Y(meta/tile)=0xA000/0x980000 size UV(meta/tile)=0x0/0x0	0x00C TILE_CFG1 = 0x00000400 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00001000 0x01C OTF_CFG1 = 0x00041020 0x020 OTF_CFG2 = 0x10000008 0x024 OTF_CFG3 = 0x00020264 0x028 OTF_CFG4 = 0x02580004 0x02C META_CFG0 = 0x00980100
YUV420_8 / NV12	128x128	tile=32x8, bpp=1 Y meta cols/rows=4/16 UV meta cols/rows=4/8 stored Y/UV=128/64 tile_pitch=8, meta_pitch=64 size Y(meta/tile)=0x1000/0x4000 size UV(meta/tile)=0x1000/0x2000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x00000008 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00020080 0x01C OTF_CFG1 = 0x000400A0 0x020 OTF_CFG2 = 0x00800008 0x024 OTF_CFG3 = 0x0002008C 0x028 OTF_CFG4 = 0x00800004 0x02C META_CFG0 = 0x00100004

格式	分辨率	关键计算	DEC 写值参考
YUV420_8 / NV12	544x1200	tile=32x8, bpp=1 Y meta cols/rows=20/152 UV meta cols/rows=20/76 stored Y/UV=1216/608 tile_pitch=40, meta_pitch=64 size Y(meta/tile)=0x3000/0xBE000 size UV(meta/tile)=0x2000/0x5F000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x00000028 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00020220 0x01C OTF_CFG1 = 0x00040240 0x020 OTF_CFG2 = 0x02200008 0x024 OTF_CFG3 = 0x000204BC 0x028 OTF_CFG4 = 0x04B00004 0x02C META_CFG0 = 0x00980014
YUV420_8 / NV12	1920x1080	tile=32x8, bpp=1 Y meta cols/rows=60/136 UV meta cols/rows=60/68 stored Y/UV=1088/544 tile_pitch=120, meta_pitch=64 size Y(meta/tile)=0x3000/0x1FE000 size UV(meta/tile)=0x2000/0xFF000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x00000078 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00020780 0x01C OTF_CFG1 = 0x000407A0 0x020 OTF_CFG2 = 0x07800008 0x024 OTF_CFG3 = 0x00020444 0x028 OTF_CFG4 = 0x04380004 0x02C META_CFG0 = 0x0088003C

格式	分辨率	关键计算	DEC 写值参考
YUV420_8 / NV12	4096x600	tile=32x8, bpp=1 Y meta cols/rows=128/76 UV meta cols/rows=128/40 stored Y/UV=608/320 tile_pitch=256, meta_pitch=128 size Y(meta/tile)=0x3000/0x260000 size UV(meta/tile)=0x2000/0x140000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x00000100 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00021000 0x01C OTF_CFG1 = 0x00041020 0x020 OTF_CFG2 = 0x10000008 0x024 OTF_CFG3 = 0x00020264 0x028 OTF_CFG4 = 0x02580004 0x02C META_CFG0 = 0x004C0080
YUV420_10 / P010	128x128	tile=32x4, bpp=2 Y meta cols/rows=4/32 UV meta cols/rows=4/16 stored Y/UV=128/64 tile_pitch=16, meta_pitch=64 size Y(meta/tile)=0x1000/0x8000 size UV(meta/tile)=0x1000/0x4000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x00000010 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00030080 0x01C OTF_CFG1 = 0x000400A0 0x020 OTF_CFG2 = 0x00800008 0x024 OTF_CFG3 = 0x0002008C 0x028 OTF_CFG4 = 0x00800004 0x02C META_CFG0 = 0x00200004

格式	分辨率	关键计算	DEC 写值参考
YUV420_10 / P010	544x1200	tile=32x4, bpp=2 Y meta cols/rows=20/300 UV meta cols/rows=20/152 stored Y/UV=1200/608 tile_pitch=80, meta_pitch=64 size Y(meta/tile)=0x5000/0x177000 size UV(meta/tile)=0x3000/0xBE000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x00000050 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00030220 0x01C OTF_CFG1 = 0x00040240 0x020 OTF_CFG2 = 0x02200008 0x024 OTF_CFG3 = 0x000204BC 0x028 OTF_CFG4 = 0x04B00004 0x02C META_CFG0 = 0x012C0014
YUV420_10 / P010	1920x1080	tile=32x4, bpp=2 Y meta cols/rows=60/272 UV meta cols/rows=60/136 stored Y/UV=1088/544 tile_pitch=240, meta_pitch=64 size Y(meta/tile)=0x5000/0x3FC000 size UV(meta/tile)=0x3000/0x1FE000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x000000F0 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00030780 0x01C OTF_CFG1 = 0x000407A0 0x020 OTF_CFG2 = 0x07800008 0x024 OTF_CFG3 = 0x00020444 0x028 OTF_CFG4 = 0x04380004 0x02C META_CFG0 = 0x0110003C

格式	分辨率	关键计算	DEC 写值参考
YUV420_10 / P010	4096x600	tile=32x4, bpp=2 Y meta cols/rows=128/152 UV meta cols/rows=128/76 stored Y/UV=608/304 tile_pitch=512, meta_pitch=128 size Y(meta/tile)=0x5000/0x4C0000 size UV(meta/tile)=0x3000/0x260000	0x008 TILE_CFG0 = 0x00000306 0x00C TILE_CFG1 = 0x00000200 0x010 TILE_CFG2 = 0x00000001 0x018 OTF_CFG0 = 0x00031000 0x01C OTF_CFG1 = 0x00041020 0x020 OTF_CFG2 = 0x10000008 0x024 OTF_CFG3 = 0x00020264 0x028 OTF_CFG4 = 0x02580004 0x02C META_CFG0 = 0x00980080

4.4.1 DEC 地址、pitch 和 OTF timing 计算方法

```
// 基础函数
ceil_div(a, b) = (a + b - 1) / b
align_up(a, n) = ceil_div(a, n) * n

// 格式参数
RGBA8888:          format=0, tile_w=16, tile_h=4, bpp=4
YUV420_8 / NV12:    format=2, tile_w=32, tile_h=8, bpp=1
YUV420_10 / P010:   format=3, tile_w=32, tile_h=4, bpp=2

// DEC pitch / metadata tile 数
aligned_width_px    = align_up(width_px, tile_w * 4)
meta_tile_x_numbers = ceil_div(aligned_width_px, tile_w)
stored_y_height      = align_up(height_px, tile_h * 4)
stored_uv_height     = align_up(ceil_div(height_px, 2), tile_h * 4)
meta_tile_y_numbers = ceil_div(stored_y_height, tile_h)
```

```

surface_pitch_bytes = align_up(width_px * bpp, tile_w * 4 * bpp)
tile_pitch          = surface_pitch_bytes / 16
meta_pitch          = align_up(meta_tile_x_numbers, 64)

// DEC metadata / tile size
meta_y_size         = align_up(meta_pitch * align_up(meta_tile_y_numbers, 4), tile_w * 4 * bpp)
tile_y_size         = align_up(surface_pitch_bytes * stored_y_height, 4096)
meta_uv_rows        = ceil_div(stored_uv_height, tile_h)
meta_uv_size        = 0 for RGBA; otherwise align_up(meta_pitch * align_up(meta_uv_rows, 4), tile_w * 4 * bpp)
tile_uv_size        = 0 for RGBA; otherwise align_up(surface_pitch_bytes * stored_uv_height, 4096)

// 连续输入 UBWC buffer 地址, base 是 RGBA/Y metadata 起始地址
REG_META_BASE_Y     = base
REG_TILE_BASE_Y     = REG_META_BASE_Y + meta_y_size
REG_META_BASE_UV    = 0 for RGBA; otherwise REG_TILE_BASE_Y + tile_y_size
REG_TILE_BASE_UV    = 0 for RGBA; otherwise REG_META_BASE_UV + meta_uv_size
total_size          = meta_y_size + tile_y_size + meta_uv_size + tile_uv_size

// DEC OTF timing 示例
h_total = HSA + HBP + HACT + HFP
v_total = VSA + VBP + VACT + VFP
OTF_CFG0 = {format[4:0], width_px}
OTF_CFG1 = {HSA, h_total}
OTF_CFG2 = {HACT, HBP}
OTF_CFG3 = {VSA, v_total}
OTF_CFG4 = {VACT, VBP}

```

5. ENC 使用流程

ENC 通过 IRQ_CTRL[5] 写 1 产生 START token。软件先配置好静态参数和输出 buffer 地址，写 IRQ_CTRL[5]=1 后，再送上游 OTF 输入流。

5.1 首次或格式变化时配置

```
read(0x000); // VERSION
read(0x004); // DATE
write(0x00C, REG_TILE_CFG1); // [0] four_line_format; [1] lossy_
write(0x008, REG_TILE_CFG0); // [0] ubwc_en; [3:1] bank_swizzle;
write(0x014, REG_ENC_CI_CFG1); // [16] enc_ci_lossy
write(0x018, REG_ENC_CI_CFG2); // [30:0] enc_ci_ubwc_cfg_0..9, 建议
write(0x01C, REG_ENC_CI_CFG3); // [5:0] cfg_10; [13:8] cfg_11, 建议
write(0x010, REG_ENC_CI_CFG0); // [0] input_type=1; [10:8] alen=7
write(0x024, REG_OTF_CFG1); // [15:0] width; [31:16] height
write(0x028, REG_OTF_CFG2); // [15:0] tile_w; [19:16] tile_h
write(0x02C, REG_OTF_CFG3); // [15:0] y_tile_cols; [31:16] uv_t
write(0x050, REG_META_ACTIVE_SIZE); // [15:0] active_width; [31:16] a
write(0x054, REG_META_PITCH); // [31:0] meta_data_plane_pitch
write(0x020, REG_OTF_CFG0); // [2:0] format
```

5.2 每帧地址配置

地址项	APB 地址	说明
Y metadata base	0x030 / 0x034	低 32 bit / 高 32 bit; RGBA 使用该槽位
Y tile base	0x038 / 0x03C	低 32 bit / 高 32 bit; RGBA 使用该槽位
UV metadata base	0x040 / 0x044	低 32 bit / 高 32 bit; RGBA 写 0
UV tile base	0x048 / 0x04C	低 32 bit / 高 32 bit; 写完四个地址后写 IRQ_CTRL[5] 启动本帧

写入顺序建议：按 meta Y、tile Y、meta UV、tile UV 顺序写。四个地址建议按顺序写完，然后写 `REG_IRQ_CTRL[5]=1` 启动本帧。

5.3 后续帧配置

如果格式和尺寸不变，后续帧只需要重复 5.2 的每帧地址配置，写 `REG_IRQ_CTRL[5]=1`，然后送入下一帧 OTF 输入流。

6. DEC 使用流程

DEC 需要软件显式写 START。首次配置静态参数和一组输入 UBWC buffer 地址后，写 `IRQ_CTRL[5]=1` 启动；后续帧继续写入新的输入 buffer 地址并再次写 START。

6.1 首次或格式/timing 变化时配置

```
read(0x000); // VERSION
read(0x004); // DATE
write(0x008, APB_ADDR_TILE_CFG0); // [2:0] bank_swizzle; [8:4] highest
write(0x00C, APB_ADDR_TILE_CFG1); // [11:0] tile_cfg_pitch
write(0x010, APB_ADDR_TILE_CFG2); // [0] ci_input_type; [8] ci_lossy;
write(0x014, VIVO_CFG);           // [0] vivo_ubwc_en; [1] vivo_srese
write(0x02C, META_CFG0);          // [15:0] Y/RGBA tile_x_numbers; [3
write(0x018, OTF_CFG0);           // [15:0] img_width; [20:16] format
write(0x01C, OTF_CFG1);           // [15:0] h_total; [31:16] h_sync/H
write(0x020, OTF_CFG2);           // [15:0] h_bp/HBP; [31:16] h_act/H
write(0x024, OTF_CFG3);           // [15:0] v_total; [31:16] v_sync/V
write(0x028, OTF_CFG4);           // [15:0] v_bp/VBP; [31:16] v_act/V
```

6.2 每帧地址配置

地址项	APB 地址	说明
metadata RGBA/Y base	0x030 / 0x034	低 32 bit / 高 32 bit
tile RGBA/Y base	0x038 / 0x03C	低 32 bit / 高 32 bit
metadata UV base	0x040 / 0x044	低 32 bit / 高 32 bit

地址项	APB 地址	说明
tile UV base	0x048 / 0x04C	低 32 bit / 高 32 bit；RGBA 图像不关心
START	0x060[5]	四类地址都写完后写 1；地址写入只填充队列，不单独启动帧

写入顺序建议：每个 64 bit 地址都按 low word、high word 顺序写。四类地址都写完后，该帧输入 buffer 地址配置完成；随后写 `IRQ_CTRL[5]=1` 启动本帧。后续帧如果格式和 timing 不变，只重复本节地址配置和 START。

7. DEC OTF timing 配置

软件可以按 video mode timing 图中的 HSA/HBP/HACT/HFP 和 VSA/VBP/VACT/VFP 计算当前 RTL 的 OTF_CFG。RTL APB 写像素/行单位，内部水平计数会转换为 128-bit OTF beat。

图中标记	名称	软件配置含义
HSA	Horizontal Sync Active	每一行开头的水平同步脉冲宽度，对应 <code>cfg_otf_h_sync</code> 。
HBP	Horizontal Back Porch	HS 结束后到 active pixel 开始前的水平空白区，对应 <code>cfg_otf_h_bp</code> 。
HACT	Horizontal Active	一行内有效显示/有效数据宽度，对应 <code>cfg_otf_h_act</code> ，通常等于输出图像宽度。
HFP	Horizontal Front Porch	active pixel 结束后到下一行 HS 前的水平空白区，不单独配置，由 <code>h_total</code> 减去 HSA/HBP/HACT 得到。
VSA	Vertical Sync Active	一帧开头的垂直同步行数，对应 <code>cfg_otf_v_sync</code> 。
VBP	Vertical Back Porch	VS 结束后到 active line 开始前的垂直空白区，对应 <code>cfg_otf_v_bp</code> 。

图中标记	名称	软件配置含义
VACT	Vertical Active	一帧内有效显示/有效数据行数，对应 <code>cfg_otf_v_act</code> ，通常等于输出图像高度。
VFP	Vertical Front Porch	<code>active line</code> 结束后到下一帧 VS 前的垂直空白区，不单独配置，由 <code>v_total</code> 减去 <code>VSA/VBP/VACT</code> 得到。

RTL 配置项	video timing 对应项	说明
<code>cfg_otf_h_sync</code>	HSA	水平 sync 宽度
<code>cfg_otf_h_bp</code>	HBP	水平 back porch
<code>cfg_otf_h_act</code>	HACT	水平 active 宽度
<code>cfg_otf_h_total</code>	$HSA + HBP + HACT + HFP$	水平总长度
<code>cfg_otf_v_sync</code>	VSA	垂直 sync 宽度
<code>cfg_otf_v_bp</code>	VBP	垂直 back porch
<code>cfg_otf_v_act</code>	VACT	垂直 active 高度
<code>cfg_otf_v_total</code>	$VSA + VBP + VACT + VFP$	垂直总行数

```
write(0x01C, {HSA, HSA + HBP + HACT + HFP});
write(0x020, {HACT, HBP});
write(0x024, {VSA, VSA + VBP + VACT + VFP});
write(0x028, {VACT, VBP});
```

当前 `ubwc_dec_otf_driver.v` 中 `hsync` 只在 active vertical 区间内输出。如果要严格模拟标准 display timing，需要让 `hsync` 在每一行都按 HSA 输出。

8. 状态、中断和统计

类别	地址	说明
ENC STATUS0	0x058	idle/error/busy/frame_done/addr_cfg_invalid/addr_cfg_valid0/1
ENC IRQ_CTRL	0x060	irq_enable、irq_clear、start、irq_pending、correct/error pending
ENC 统计	0x068..0x09C	metadata、tileaddr、OTF、AXI 写计数
DEC STATUS0	0x050	frame_active、meta/tile/vivo/otf busy、idle 状态
DEC IRQ_CTRL	0x060	irq_enable、irq_clear、start、irq_pending、error/correct pending
DEC 统计	0x068..0x078	metadata tile、tileaddr、OTF tile、line、de beat 计数

```
// ENC 清中断
write(0x060, old_irq_enable | (1 << 1));

// DEC 清中断
write(0x060, old_irq_enable | (1 << 1));
```

9. 当前回归基线

2026-05-17 在服务器

10.168.1.199:/home/eda/work/ubwc/regress_sram_split_20260517 完成 4096 宽、COM_BUF_AW=12、两帧 VCS fake 回归。

```
make -C vrf/sim TOP=tb_ubwc_dec_wrapper_top_tajmahal_4096x600_nv12 CA
make -C vrf/sim TOP=tb_ubwc_dec_wrapper_top_tajmahal_4096x600_nv12 TE
make -C vrf/sim TOP=tb_ubwc_dec_wrapper_top_k_outdoor61_4096x600_g016
make -C vrf/sim TOP=tb_ubwc_dec_wrapper_top_k_outdoor61_4096x600_g016
make -C vrf/sim TOP=tb_ubwc_enc_wrapper_top_tajmahal_4096x600_nv12 RL
make -C vrf/sim TOP=tb_ubwc_enc_wrapper_top_tajmahal_4096x600_nv12 RL
```

```
make -C vrf/sim TOP=tb_ubwc_enc_wrapper_top_k_outdoor61_4096x600_g016
make -C vrf/sim TOP=tb_ubwc_enc_wrapper_top_k_outdoor61_4096x600_g016
```

Case	结果	说明
tb_ubwc_dec_wrapper_top_tajmahal_4096x600_nv12 +tb_frame_repeat=2	PASS	DEC NV12 fake VIVO，两帧连续数据
tb_ubwc_dec_wrapper_top_k_outdoor61_4096x600_g016 +tb_frame_repeat=2	PASS	DEC G016/P010 fake VIVO，两帧连续数据
tb_ubwc_enc_wrapper_top_tajmahal_4096x600_nv12 +tb_frame_repeat=2	PASS	ENC NV12 fake，两帧连续数据
tb_ubwc_enc_wrapper_top_k_outdoor61_4096x600_g016 +tb_frame_repeat=2	PASS	ENC G016/P010 fake，两帧连续数据

本轮重点验证 ENC/DEC YUV420 与 RGBA 在 4096 宽、每个 bank 固定 4096 x 128 bit 时的地址、layout、AXI 访问、OTF 和连续帧行为；上述 case 最终均为 PASS。